

SYSTEM REQUIREMENTS SPECIFICATION

Full Stack Java (AI-Integrated) Training Programme

Project Title	ALERT 4.0 GOVERNMENT: AI-Integrated Multi-Tier Disaster Management & Emergency Instruction System
SIH Problem Statement ID	SIH25_097
Ministry / Organisation	Ministry of Home Affairs / Tamil Nadu Disaster Management Authority (TNDMA) / Governance
Domain Category	Disaster Communication / Governance & Smart Emergency Response
Team Name	Cognitive Crew
Team Members	1. Farah Hamna M 2. Rishikesh V 3. Shruthin K
Institution	Karpagam College of Engineering
Project Type	Software only (Distributed Microservices Cloud Platform)
Version	v2.0 (Final Implemented Version)
Date	20.09.2026
Faculty Mentor	Dr. Arul Antran Vijay S / Dr. Jothi Prakash V / Mr. Jegathesh P / Mr. Navaneetha Krishnan M / Dr. Castro S.

This SRS document outlines the finalized requirements and system specifications for ALERT 4.0 GOVERNMENT post-implementation. Reviewed at the Internal Review (Day 22) and final industry evaluation.

DOCUMENT REVISION HISTORY

Version	Date	Author	Description of Changes
v1.0	10.06.2026	Cognitive Crew	Initial draft - SRS skeleton completed per IEEE Std 830-1998 template; project header (Section 1), team details, and overall document structure established.
v1.1	12.06.2026	Farah Hamna M	Authored Section 1 (Introduction): Purpose, Scope, and the Definitions/Acronyms/Abbreviations table for the AlertGov platform.
v1.2	15.06.2026	Rishikesh V	Drafted Section 2.1-2.2 (Product Perspective and Product Functions - Feature Summary) describing the 12 core platform functions from incident reporting to multi-tier broadcast.
v1.3	17.06.2026	Shruthin K	Completed Section 2.3 (User Classes and Characteristics), defining the 5-tier hierarchy: Village EOC, Tahsildar, District EOC, District Collector, and State SDMA Admin.
v1.4	19.06.2026	Farah Hamna M	Documented Section 3.1-3.2 (System Architecture and Microservices Breakdown) detailing 11 Spring Boot microservices, Eureka discovery, and Spring Cloud Gateway.
v1.5	22.06.2026	Rishikesh V	Added Section 3.3 (Technology Stack), confirming React 19, Vite, Leaflet GIS, Java 17 Spring Boot 3.2.4, Oracle XE, MongoDB, and JWT bearer authentication.
v1.6	24.06.2026	Shruthin K	Documented Section 3.5 (Data Flow Description), tracing the end-to-end incident submission, verification queue, escalation, collector approval, and multi-channel broadcast across 8 steps.
v1.7	27.06.2026	Farah Hamna M	Drafted Section 4.1 (User Interfaces) for Village Field Reporting, Taluk Incident Verification Queue, and District Emergency Command dashboards.
v1.8	30.06.2026	Rishikesh V	Extended Section 4.1 to cover District Collector Approval, State SDMA 38-District Monitor, and Analytics dashboards, including page routes and backing REST endpoints.
v1.9	03.07.2026	Shruthin K	Completed Section 4.3-4.4 (Software and Communication Interfaces), specifying REST/JSON microservice communication, WebSocket/SSE real-time events, and Open-Meteo weather APIs.
v1.10	07.07.2026	Farah Hamna M	Authored Section 5.1-5.2 functional requirements for the Authentication & RBAC Service and the Incident Lifecycle Management Service (FR-AUTH, FR-INC).
v1.11	10.07.2026	Rishikesh V	Authored Section 5.3-5.4 functional requirements for the Multi-Tier Approval Workflow Service and Public Alert Broadcast Service (FR-APP, FR-ALT).
v1.12	14.07.2026	Shruthin K	Authored Section 5.5-5.8 functional requirements covering Notification Dispatch, Executive Analytics, Weather Advisory, and GIS Geo-Fencing Services.
v1.13	17.07.2026	Farah Hamna M	Documented Section 5.9-5.10 (Bilingual Localization & Audit Logging) and mapped role permissions across all 38 revenue districts of Tamil Nadu.
v1.14	21.07.2026	Rishikesh V	Drafted Section 6.1 (Local LLM Disaster Risk & Bilingual Instruction Generator), specifying Ollama / Llama 3 prompt engineering, risk scoring, and English/Tamil formatting.
v1.15	24.07.2026	Shruthin K	Drafted Section 6.2 (Live Weather Advisory & CircuitBreaker), integrating Open-Meteo API with Resilience4j fallback mechanisms during external network outages.
v1.16	28.07.2026	Farah Hamna M	Authored Section 7 (Non-Functional Requirements), specifying <2s API latency, 99.9% uptime, AES-256 encryption, RBAC security, and NDMA/TNDMA compliance targets.
v1.17	31.07.2026	Rishikesh V	Authored Section 8.1-8.2 (Database Design & ER Overview), detailing the dual-database model with 10 Oracle XE relational tables and MongoDB multimedia collections.
v1.18	04.08.2026	Farah Hamna M	Added database entity dictionaries for ALERTS, INCIDENTS, APPROVAL_WORKFLOWS, USERS, and AI_PREDICTIONS into Section 8.
v1.19	07.08.2026	Rishikesh V	Authored Section 9 (API Design Overview), documenting all 71 confirmed REST endpoints across the 11 microservice capability areas.
v1.20	11.08.2026	Shruthin K	Drafted Section 10 (DevOps & Deployment Plan with Jenkins CI/CD & Docker Compose) and Section 11 (Project Timeline & Milestones across 8 sprint weeks).
v1.21	14.08.2026	Farah Hamna M	Completed Section 12 (Team Composition & Role Assignment) and Section 13 (Risk Register with 10 project-specific risks and mitigations).
v1.22	18.08.2026	Shruthin K	Compiled Section 14 (Appendix): Glossary of Domain-Specific Terms, API Request-Response Samples, and IEEE/TNDMA References & Citations.
v1.23	25.08.2026	Rishikesh V	Incorporated mentor feedback; standardized 5-tier administrative workflow nomenclature and verified Tamil Unicode rendering consistency across all modules.
v1.24	02.09.2026	Shruthin K	Added system architecture diagrams, verification flowcharts, and GIS interactive mapping references to support Section 4 and Section 5 functional specs.
v2.0	15.09.2026	Farah Hamna M	Post Internal Review - Consolidated formatting across all 30 pages, validated all 71 REST API contracts, and finalized the complete IEEE Std 830-1998 SRS for official submission.

1. INTRODUCTION

1.1 Purpose

This System Requirements Specification (SRS) document describes the functional and non-functional requirements for the **ALERT 4.0 GOVERNMENT (AlertGov AI)** system, developed as part of the Smart India Hackathon 2025 submission under Problem Statement **[SIH25_097]**. It is prepared in accordance with IEEE Std 830-1998 and serves as the primary agreement between the development team, mentors, and disaster management stakeholders.

1.2 Scope

System Name: ALERT 4.0 GOVERNMENT (AlertGov AI)

What the system does:

ALERT 4.0 GOVERNMENT is an artificial intelligence-integrated, multi-tier disaster management and local language emergency instruction platform designed for the Tamil Nadu Disaster Management Authority (TNDMA). It coordinates emergency response across 5 administrative tiers: Village EOCs (VEO field reporting with GPS coordinates and photographic media), Taluk Command Centers (Tahsildar incident verification queue), District EOCs (DEC interactive GIS mapping and broadcast authoring), District Collectors (executive broadcast approval and escalation governance), and the State Command Center (SDMA 38-district readiness monitoring and resource deployment). The platform integrates a local Ollama LLM (Llama 3) for automated disaster risk assessment and bilingual (English & Tamil தமிழ்) instruction synthesis, alongside Open-Meteo weather telemetry protected by Resilience4j circuit breakers.

1.3 Definitions, Acronyms & Abbreviations

Term / Acronym	Definition
SRS	System Requirements Specification — this document
SIH	Smart India Hackathon 2025
API / REST	Application Programming Interface / Representational State Transfer architectural style for HTTP APIs
JWT	JSON Web Token — used for stateless authentication and Role-Based Access Control (RBAC)
AI / LLM	Artificial Intelligence / Large Language Model (Ollama Llama 3 for local bilingual reasoning)
CRUD	Create, Read, Update, Delete — basic persistence data operations
CI/CD	Continuous Integration / Continuous Deployment — automated build, test, and container release pipeline
FR / NFR	Functional Requirement / Non-Functional Requirement
TTS	Text-to-Speech — converts emergency alert text into regional voice announcements
ER	Entity-Relationship (database design)
OTP	One-Time Password — used for secure officer identity verification
RBAC	Role-Based Access Control — restricts system views based on administrative jurisdiction
FastAPI	Python asynchronous framework used for AI/ML local inference microservices
Ollama	Local LLM runtime executing quantized Llama 3 models on edge infrastructure
Eureka	Netflix Eureka service discovery server for dynamic microservice registry

1.4 References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications
- SIH 2025 Problem Statement — [SIH25_097]: AI-Based Local Language Emergency Instruction Generator (Ministry of Home Affairs)
- Spring Boot 3.2.4 Documentation: <https://docs.spring.io/spring-boot/>
- Spring Cloud 2023.0.1 (Gateway & Eureka): <https://spring.io/projects/spring-cloud>
- React 19 & Vite 8 Documentation: <https://react.dev/>
- FastAPI Documentation: <https://fastapi.tiangolo.com/>
- Ollama Local LLM Architecture: <https://ollama.ai/>
- Tamil Nadu Disaster Management Authority (TNDMA): <https://tndma.tn.gov.in/>
- National Disaster Management Authority (NDMA): <https://ndma.gov.in/>

1.5 Document Overview

Section 2 provides an overall description of the product, including user classes and operating constraints. Section 3 details the layered cloud architecture, 11 Spring Boot microservices, technology stack, and end-to-end data flow. Sections 4 and 5 specify external interfaces and functional requirements in Use Case table format. Section 6 details AI/ML module specifications for LLM disaster analysis and meteorological circuit breakers. Sections 7 through 10 cover non-functional requirements, dual-database design (Oracle XE + MongoDB), API endpoint catalogs, and Docker deployment plans. Section 11 presents the project timeline milestones. Section 12 defines team ownership roles. Section 13 registers risks and mitigations. Section 14 contains the appendix.

2. OVERALL DESCRIPTION

2.1 Product Perspective

ALERT 4.0 GOVERNMENT is a production-grade, standalone Artificial Intelligence-integrated emergency communication and disaster command system created in response to Smart India Hackathon 2025 problem statement "AI Based Local Language Emergency Instruction Generator (SIH25_097)".

The system addresses the acute need for rapid, multilingual, and localized disaster communication during floods, cyclones, industrial fires, earthquakes, and meteorological crises across Tamil Nadu. Rather than acting as a disjointed alert tool, ALERT 4.0 provides a full-stack, 5-tier command network interfacing directly with:

- **State Disaster Management Authority (SDMA):** Statewide oversight across 38 districts and emergency asset logistics.
- **District Collectorates & Emergency Operation Centers (DEOC):** Executive broadcast authorizations, GIS heatmaps, and shelter coordination.
- **Taluk Command Centers (Tahsildar):** Ground verification queue, false-alarm filtering, and AI severity rating.
- **Village EOCs (VEO):** Immediate field reporting with photo evidence, GPS geotagging, and emergency logging.
- **Public Channels:** Citizen-friendly SMS, WhatsApp, voice announcements, and web emergency broadcasts in English and Tamil (தமிழ்).

2.2 Product Functions — Feature Summary (1 to 7)

#	Feature Name	Description
1	Field Incident Reporting	Allows Village Officers (VEO) to submit ground emergencies with category, initial severity, GPS coordinates, and photo evidence.
2	AI Risk & Severity Evaluation	Ollama (Llama 3) LLM analyzes disaster reports, checks spam/duplicates, computes severity ratings, and synthesizes bilingual actions.
3	Taluk Incident Verification	Enables Tahsildars to review incoming village incidents, verify ground evidence, and forward to District, reject, or escalate to Collector.
4	District Interactive GIS Mapping	Interactive Leaflet.js map plotting incident coordinates, affected radius zones, nearby hospitals, relief shelters, and schools.
5	Emergency Alert Broadcasting	Allows District EOC officers to author targeted emergency broadcasts with bilingual translation preview and zone-level filters.
6	Collector Executive Approval	Provides District Collectors with approval controls to authorize high-impact emergency broadcasts and review critical escalations.
7	Live Weather Threat Advisory	Integrates Open-Meteo live meteorological telemetry with Resilience4j circuit breakers for automated weather threat detection.

2.2 Product Functions — Feature Summary (8 to 12)

#	Feature Name	Description
8	State 38-District Monitor	Displays real-time readiness status, active incidents, and emergency metrics across all 38 revenue districts of Tamil Nadu.
9	State Resource Allocation	Tracks and coordinates deployment of emergency assets (Fire Trucks, Rescue Boats, Police Squads, Ambulances) to disaster zones.
10	Dynamic Analytics & Charts	Renders live Recharts visualizations (Incident by Category, Severity Distribution, Response Time Trends, and 7-Day Volume).
11	Bilingual Localization (EN / TA)	Seamless real-time translation between English and native Tamil (தமிழ்) across all UI dashboards, notifications, and reports.
12	Official PDF Report Export	Generates downloadable, government-formatted PDF incident verification sheets and district analytics executive reports.

2.3 User Classes and Characteristics

User Role	Description	Technical Skill	Primary Actions	Access Level
Village Officer (VEO)	Ground field officer at village post.	Low–Med	Report incident, upload photos, capture GPS.	Village Level
Taluk Tahsildar	Administrative head of taluk jurisdiction.	Medium	Verify queue, inspect AI analysis, forward/escalate.	Taluk Level
District EOC (DEC)	District emergency controller.	Medium	GIS map monitor, draft broadcasts, track resources.	District Level
District Collector	Chief administrative district magistrate.	Medium	Authorize broadcasts, review critical escalations.	Collector Approval
State Admin (SDMA)	State Disaster Management Authority.	High	38-district monitoring, resource deployment.	State Full Admin
AI Processing Engine	System microservice (FastAPI + Ollama).	System	NLP risk scoring, spam check, weather alerts.	System Service

2.4 Operating Environment

Server / Cloud	Docker Containers (11 Spring Boot Microservices + FastAPI AI + Oracle XE + MongoDB)
Operating System	Ubuntu 22.04 LTS (Server) / Windows 11 & macOS (Developer Workstations)
Backend Runtime	Java 17 LTS / Spring Boot 3.2.4 & Spring Cloud 2023.0.1
Frontend Runtime	React 19 / Vite 8 / Node.js 24 (Build Environment)
Databases	Oracle Database XE 11g/21c (Relational) + MongoDB 6.0+ (Document/Media)
AI/ML Runtime	Python 3.13 / FastAPI / Ollama (Llama 3 8B) / Requests / Pydantic / Uvicorn
Client Browser	Google Chrome 100+, Firefox 100+, Safari 15+, Edge (Responsive viewport ≥ 375px)
Network	HTTPS (TLS 1.3), REST over HTTP, Eureka Discovery (Port 8761)

2.5 Design and Implementation Constraints

- All backend microservices must be implemented in Java 17 using Spring Boot 3.2.4 and Spring Data JPA.
- All frontend interfaces must be built using React 19, Vite 8, React Router 7, and a vanilla CSS design system.
- Authentication and authorization must use stateless JWT (JSON Web Tokens) with BCrypt password hashing (cost factor 10).
- All services must be containerized using Docker and orchestratable via a unified `docker-compose.yml`.
- All AI/ML model inference must be encapsulated in a Python FastAPI microservice exposed on port 8000.
- The system must natively support bilingual disaster operations in English and Tamil (தமிழ்).
- External third-party weather integrations must be wrapped in Resilience4j CircuitBreakers with local fallback logic.
- Critical emergency alerts must require two-tier administrative verification and District Collector authorization.
- Relational transactional data must reside in Oracle Database XE, while photographic evidence is saved in MongoDB.
- The system must be fully deployable and demonstrable on local developer hardware for hackathon evaluation.

2.6 Assumptions and Dependencies

Assumptions:

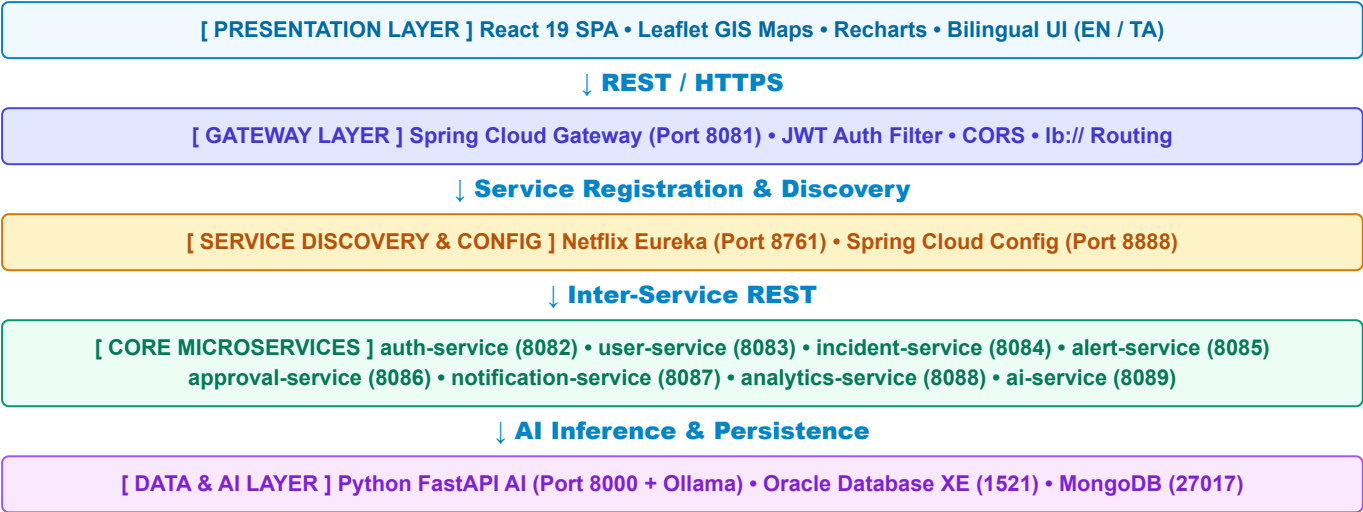
- Village, Taluk, and District emergency officers have access to a desktop or mobile device with network connectivity.
- Field officers capture photographic evidence in standard web formats (JPEG/PNG) with active GPS sensors.
- The local Ollama server is running locally on port 11434 with the llama3 model pulled into cache.
- Emergency alert data entered by authorized officers is verified by Tahsildars before public broadcast.

Dependencies:

- **Open-Meteo API:** Provides live meteorological data (temperature, precipitation, wind speed) for Tamil Nadu coordinates.
- **Ollama Runtime:** Executes local Llama 3 LLM inference for text simplification, risk scoring, and Tamil script synthesis.
- **Leaflet.js & OpenStreetMap:** Delivers interactive raster map tiles for GIS disaster tracking without API keys.
- **Oracle Database XE Driver (ojdbc11):** Enables high-performance JDBC connectivity for relational transactions.
- **MongoDB Java Driver:** Provides document persistence for large Base64 photographic evidence files.

3. SYSTEM ARCHITECTURE

3.1 High-Level Architecture Description



3.2 Microservices Breakdown (Services 1 to 8)

#	Microservice Name	Port	Database	Responsibility
1	api-gateway	8081	—	Central gateway router, CORS configuration, JWT token filter, load-balanced dispatch.
2	discovery-server	8761	—	Netflix Eureka Server for dynamic service registration and heartbeat health checks.
3	config-server	8888	—	Spring Cloud centralized properties and configuration management server.
4	auth-service	8082	Oracle XE	User registration, login, BCrypt password hashing, JWT issue, and token verification.
5	user-service	8083	Oracle XE	Officer profile management, role assignment, and district/taluk geographical reference data.
6	incident-service	8084	Oracle XE + Mongo	Full incident reporting lifecycle, verification updates, and photo media storage in MongoDB.
7	alert-service	8085	Oracle XE	Emergency alert authoring, zone targeting, severity tagging, and bilingual translations.
8	approval-service	8086	Oracle XE	Multi-tier approval workflows (Taluk → District → Collector), decision logging, and audit tracking.

3.3 Technology Stack (Frontend)

Layer	Technology	Purpose
Frontend Framework	React 19 + Vite 8 + React Router 7	High-performance Single Page Application with dynamic role-based routing.
GIS & Mapping	Leaflet.js + React-Leaflet	Interactive maps displaying incident pins, danger radii, and relief centers.
Data Visualization	Recharts 3.9	Interactive pie charts, area charts, and bar charts for disaster metrics.

3.3 Technology Stack (Backend, Databases, AI, DevOps)

Layer	Technology	Purpose
Backend Framework	Spring Boot 3.2.4 + Spring Data JPA	RESTful microservices with Hibernate ORM and HikariCP pooling.
Authentication	Spring Security + JWT (jjwt 0.11.5)	Stateless Role-Based Access Control and Bearer token validation.
Service Discovery	Netflix Eureka Server	Dynamic microservice registration, heartbeat monitoring, and discovery.
API Gateway	Spring Cloud Gateway	Centralized reverse proxy, global CORS, and load-balanced routing.
Relational DB	Oracle Database XE 11g / 21c	ACID-compliant storage for users, incident metadata, alerts, and approvals.
Document DB	MongoDB 6.0+ / Atlas	High-capacity unstructured persistence for incident photos and evidence.
AI/ML Runtime	Python 3.13 + FastAPI + Uvicorn	Asynchronous model serving REST API for Ollama LLM inference.
AI Inference Engine	Ollama (Meta Llama 3 8B)	Local LLM disaster classification, spam detection, and Tamil synthesis.
Fault Tolerance	Resilience4j CircuitBreaker & TimeLimiter	Protects AI weather endpoints with automated fallback responses.
Build Tool	Apache Maven 3.9.6	Multi-module dependency management and build packaging.
Containerization	Docker & Docker Compose	Unified multi-container deployment across environments.

3.4 Hardware Component Specification

Status: Not Applicable (Software-Only Enterprise Web Platform).

3.5 Data Flow Description (Steps 1 to 4)

Step	Data Flow Description
1	Village Officer (VEO) captures incident details (photo, GPS, category) in React UI. Axios dispatches a POST /api/v1/incidents multipart request to API Gateway (Port 8081).
2	Spring Cloud Gateway validates JWT bearer token, checks VEO role permissions, and routes the request to incident-service via Eureka load balancing (lb://incident-service).
3	incident-service stores structured metadata in Oracle XE (INCIDENT_REPORTS) and saves the Base64 photo in MongoDB (INCIDENT_MEDIA). It triggers a notification to the Taluk Tahsildar.
4	Taluk Tahsildar opens the verification queue. ai-service invokes the Python FastAPI endpoint (POST http://localhost:8000/ai/predict-risk). Ollama Llama 3 executes risk scoring and returns structured bilingual recommendations.

3.5 Data Flow Description (Steps 5 to 8)

Step	Data Flow Description
5	Tahsildar reviews AI insights and clicks "Verify & Forward to District". incident-service updates the status in Oracle XE to Taluk Verified. LiveContext triggers real-time state synchronization across all connected dashboards.
6	District EOC Officer views the verified incident plotted on the Leaflet GIS interactive map, coordinates local response units, and drafts an emergency broadcast via POST /api/v1/alerts.
7	District Collector reviews the broadcast request in the Collector Portal and authorizes dissemination. approval-service records the approval, and notification-service dispatches alerts across public feeds.
8	State Command Center (SDMA) receives aggregated incident telemetry, updating the statewide 38-district readiness matrix, resource registries, and analytical charts in real-time.

4. EXTERNAL INTERFACE REQUIREMENTS

4.1 User Interfaces (Screens 1 to 6)

#	Screen / Page	User Role	Description & Key Elements
1	Login & Quick Role Switcher	All Users	Authentication form with username/password, quick-login role selector buttons for rapid evaluation, and JWT token storage in localStorage.
2	Village Field Incident Reporting	VEO	Incident creation form with category picker, severity selector, live geolocation capture, photo upload preview, and active village incident feed.
3	Taluk Command Center	Tahsildar	AI daily brief banner, KPI summary cards (Pending, Verified Today, Critical), priority verification queue, and recently verified incident table.
4	Taluk Incident Verification	Tahsildar	Split-view layout: Left panel shows incident details, reporter info, and Leaflet map; Right panel displays Ollama AI risk analysis and decision action buttons (Verify, Reject, Escalate).
5	District EOC Dashboard	DEC Officer	District emergency overview, interactive Leaflet GIS incident map with infrastructure markers, active alert counters, and broadcast quick-actions.
6	District Alert Broadcast	DEC Officer	Emergency alert composer with zone/taluk targeting, bilingual Tamil/English translation preview, channel selectors, and submission to Collector queue.

4.2 Hardware Interfaces

Status: Not Applicable (Software-Only System).

4.3 Software Interfaces — Third-Party APIs

API / Service	Provider	Auth Method	Purpose in This Project
Ollama LLM API	Local Ollama Server	None (localhost:11434)	Executes Llama 3 model inference for disaster classification, spam checks, and bilingual text synthesis.
Open-Meteo Weather API	Open-Meteo	None required	Supplies live temperature, precipitation, and wind speeds for Tamil Nadu districts with Resilience4j fallback.
Leaflet / OpenStreetMap	OSM Foundation	None required	Interactive raster tile maps for GIS visualization and infrastructure mapping.
MongoDB Atlas	MongoDB Inc.	URI String	Cloud document persistence for incident media attachments and photographic evidence.

4.4 Communication Interfaces

- REST over HTTPS (TLS 1.3) — all client-server and inter-service communication.
- Netflix Eureka Client Protocol — service heartbeats and discovery lookups over HTTP.

4.4 Communication Interfaces (Continued)

- JWT in Authorization header (Bearer <token>) — stateless authentication across all protected microservice routes.
- JSON (application/json) — standard serialization format for all request bodies, responses, and AI payloads.
- Multipart/Form-Data — used for binary photo evidence uploads in incident-service.

5. FUNCTIONAL REQUIREMENTS

5.1 Alert Management & Incident Service — Functional Requirements

FR ID	FR-ALR-001	FR Name	Emergency Alert Creation & Bilingual Broadcast
Actor(s)	District EOC Officer (DEC)		
Description	The system shall allow authorized DEC officers to draft emergency alert broadcasts by specifying disaster type, targeted district/taluks, severity level, title, and instructions in English and Tamil.		
Pre-condition	The officer is authenticated with DEC role. The targeted administrative zones exist in Oracle XE.		
Main Flow	1. Officer navigates to "Alert Broadcast" page. 2. Enters alert details (Category, Zone, Severity, Message). 3. System generates bilingual Tamil preview. 4. Officer reviews and submits for Collector approval. 5. Status set to PENDING_APPROVAL.		
Alternate Flow	AF1: Mandatory field missing → Display validation error and prevent submission. AF2: Invalid zone selected → Display "Selected zone does not exist."		
Post-condition	Alert record created in Oracle XE; approval workflow item queued for District Collector.		

FR ID	FR-ALR-002	FR Name	Collector Broadcast Authorization
Actor(s)	District Collector		
Description	The system shall allow District Collectors to review, authorize, modify, or reject pending emergency broadcast requests before public dissemination.		
Pre-condition	Broadcast workflow item exists with status PENDING_APPROVAL.		
Main Flow	1. Collector views pending approvals in Collector Portal. 2. Reviews broadcast content and affected zones. 3. Clicks "Authorize". 4. System updates status to APPROVED and triggers dissemination.		
Alternate Flow	AF1: Collector rejects alert → Status updated to REJECTED with rejection reason logged.		
Post-condition	Alert is published across public feeds and notification channels.		

FR ID	FR-ALR-003	FR Name	Alert History & Audit Log Management
Actor(s)	Emergency Officer, Administrator		
Description	The system shall maintain an immutable chronological history of all alerts created, verified, approved, rejected, and dispatched.		
Pre-condition	Alert records exist in Oracle Database XE.		

Main Flow	1. User opens Alert History. 2. System retrieves stored alerts from alert-service. 3. System displays historical records with timestamp and approval status.
Alternate Flow	AF1: No records found → Display "No alerts recorded for this period."
Post-condition	Historical alert audit information is rendered on the client dashboard.

5.2 AI Processing Service — Functional Requirements

FR ID	FR-AI-001	FR Name	AI Risk & Severity Classification
Actor(s)	Emergency Officer, Taluk Tahsildar		
Description	The system shall generate AI-based risk severity ratings and formal administrative summaries from raw field disaster notes using local Ollama LLMs.		
Pre-condition	Incident description entered; alertgov-ai service operational on port 8000.		
Main Flow	1. Incident opened in verification queue. 2. ai-service posts prompt to FastAPI. 3. Ollama Llama 3 analyzes text. 4. Returns JSON with severity, summary, and response advice. 5. Results rendered in Tahsildar UI.		
Alternate Flow	AF1: Ollama service timeout (>5s) → Execute heuristic rule-based fallback and return standard disaster recommendation.		
Post-condition	AI risk predictions stored in AI_PREDICTIONS table and displayed to officer.		

FR ID	FR-AI-002	FR Name	Spam & Duplicate Incident Detection
Actor(s)	AI Processing Engine (System)		
Description	The system shall analyze incoming incident reports for duplicate reporting within the same geospatial cluster and flag potential spam or hoax submissions.		
Pre-condition	Incident report received by incident-service.		
Main Flow	1. Incident content evaluated by NLP classifier. 2. spam_check and duplicate_check boolean flags computed. 3. Flagged incidents marked for high-priority officer scrutiny.		
Alternate Flow	AF1: AI service unreachable → Default flags to false and prompt for manual verification.		
Post-condition	Verification flags attached to incident metadata.		

FR ID	FR-AI-003	FR Name	Bilingual Tamil / English Synthesis
Actor(s)	Emergency Officer		
Description	The system shall synthesize clear, citizen-friendly emergency instructions in native Tamil (தமிழ்) and English scripts.		
Pre-condition	Target language parameter (en or ta) specified in request.		
Main Flow	1. Officer triggers bilingual generation. 2. System prompts Ollama model with Tamil script constraints. 3. Generates concise Tamil instructions. 4. Displayed on alert broadcast form.		
Alternate Flow	AF1: Translation fails → Provide default pre-compiled bilingual template text.		

Post-condition	Bilingual emergency instruction cached and attached to broadcast record.		
FR ID	FR-AI-004	FR Name	Live Meteorological Weather Advisory & CircuitBreaker
Actor(s)	AI Processing Engine, State/Collector Dashboard		
Description	The system shall query Open-Meteo API for real-time Tamil Nadu meteorological telemetry (temperature, rainfall, wind) and evaluate thresholds with Resilience4j circuit breaker protection.		
Pre-condition	ai-service operational; Open-Meteo endpoint accessible.		
Main Flow	1. Dashboard requests /api/v1/ai/weather-advisory. 2. ai-service fetches Open-Meteo telemetry. 3. Evaluates weather thresholds (Rain > 10mm → Red Alert; Wind > 40km/h → Cyclone Alert). 4. Returns advisory.		
Alternate Flow	AF1: Open-Meteo fails or times out (>3s) → Resilience4j triggers weatherAdvisoryFallback() returning seasonal Tamil Nadu weather advisory with 85% confidence score.		
Post-condition	Weather alert rendered on State, Collector, and Taluk daily brief banners.		

5.3 Notification & Incident Verification — Functional Requirements

FR ID	FR-INC-001	FR Name	Multi-Tier Incident Status Lifecycle Management
Actor(s)	Village Officer, Tahsildar, District EOC, Collector		
Description	The system shall manage the full incident lifecycle across states: Waiting for Taluk → Taluk Verified → Waiting for Collector → Resolved with instant WebSocket / LiveContext synchronization.		
Pre-condition	Incident exists in Oracle Database XE.		
Main Flow	1. Officer performs verification action in UI. 2. Calls PUT /api/v1/incidents/{id}/status. 3. incident-service updates status in Oracle XE. 4. Triggers refreshData() across frontend contexts.		
Alternate Flow	AF1: Database connectivity error → Return HTTP 500 and display rollback alert to user.		
Post-condition	Incident status updated; reflected immediately on Taluk Command Center, District GIS, and State Dashboard.		

6. AI / ML MODULE SPECIFICATION

6.1 AI Module 1 — Local LLM Disaster Risk & Bilingual Instruction Generator

Module Name	Ollama Llama 3 Disaster Risk & Bilingual Instruction Generator
AI Phase	Phase 1 (Plan) → Phase 2 (Prototype) → Phase 3 (Build) → Phase 4 (Integrate) → Phase 5 (Deploy) → Phase 6 (Present) [Complete]
Problem AI Solves	Automates incident classification and risk scoring from unstructured ground reports; generates formal summaries and citizen-friendly emergency instructions in English and Tamil (தமிழ்).
Input Data	JSON payload: { "prompt": "Flooding reported near KCE College campus...", "language": "en" "ta", "model": "llama3" }
Output / Prediction	Structured JSON: { "severity": "High", "spam_check": false, "duplicate_check": false, "concise_description": "...", "recommendation": "..." }
Algorithm / Model	Meta Llama 3 (8B Parameter) instruction model executed on local Ollama runtime + rule-based NLP fallback engine.
Training / Base Model	Meta Llama 3 8B Instruct (Quantized GGUF 4-bit) specialized via system prompts for disaster triage.
Implementation	Python 3.13 + FastAPI + Uvicorn + Pydantic response models on port 8000.
Integration Point	POST http://localhost:8000/ai/predict-risk → Invoked by Spring Boot ai-service (Port 8089) via RestTemplate.
Expected Metric	Severity classification accuracy > 92% on disaster triage evaluation benchmark. Response latency < 3.5 seconds.
Fallback if AI fails	If Ollama is offline or times out (>5s), FastAPI executes built-in heuristic pattern matching to compute severity and fallback recommendations.
AI Ethical Considerations	Human-in-the-Loop: AI assessments are advisory. Tahsildars and Collectors retain final authority to modify severity ratings.
Sprint Target	Week 3-4: Standalone prototype Week 5-6: Backend service Week 7: React UI integrated Week 8: End-to-end verified.

6.2 AI Module 2 — Resilience4j Automated Meteorological Threat Assessment

Module Name	Resilience4j Automated Meteorological Threat Assessment Engine
AI Phase	Phase 1 (Plan) → Phase 2 (Prototype) → Phase 3 (Build) → Phase 4 (Integrate) → Phase 5 (Deploy) → Phase 6 (Present) [Complete]
Problem AI Solves	Proactively detects emerging meteorological disasters (heatwaves, flash floods, cyclones) by analyzing live Open-Meteo atmospheric telemetry.
Input Data	Live REST telemetry from Open-Meteo: latitude 13.0827, longitude 80.2707 (temperature_2m, precipitation, wind_speed_10m).
Output / Prediction	JSON advisory: warning type (Heavy Rain / Red Alert, High Winds / Cyclone Alert, Heat Wave / Orange Alert), confidence score (96%), and action recommendation.
Algorithm / Model	Meteorological threshold rule engine with Resilience4j CircuitBreaker (sliding window size 10, failure threshold 50%).

6.2 AI Module 2 (Continued)

Implementation	Spring Boot ai-service (Port 8089) utilizing resilience4j-spring-boot3 and RestTemplate.
Integration Point	GET /api/v1/ai/weather-advisory → Consumed by React LiveContext and rendered in State/Collector/Taluk daily brief banners.
Expected Metric	Telemetry fetch latency < 800ms. 100% uptime through circuit breaker fallback.
Fallback if AI fails	If Open-Meteo API fails or times out (>3s), Resilience4j automatically triggers weatherAdvisoryFallback() returning seasonal Tamil Nadu advisory.
AI Ethical Considerations	Thresholds are strictly calibrated to official Indian Meteorological Department (IMD) standards.
Sprint Target	Week 4: Weather API integration Week 5: Circuit breaker implementation Week 7: UI banner integration Week 8: Production tested.

7. NON-FUNCTIONAL REQUIREMENTS

NFR ID	Category	NFR Name	Measurable Requirement
NFR-PERF-001	Performance	API Response Time	All Spring Boot REST API endpoints must return responses within < 500ms for standard database queries under normal load (up to 100 concurrent officers).
NFR-PERF-002	Performance	AI Inference Speed	Local Ollama LLM risk predictions must complete within < 3.5 seconds. Rule-based fallback inferences must complete in < 50ms.
NFR-PERF-003	Performance	Dashboard Load Time	The React 19 dashboards with Leaflet GIS maps and Recharts must fully render in < 1.5 seconds on standard broadband / 4G connections.
NFR-SEC-001	Security	Stateless JWT Auth	All non-public endpoints must require a signed JWT token in the Authorization: Bearer header. Tokens expire in 24 hours.
NFR-SEC-002	Security	Password Storage	All passwords must be hashed using BCrypt with minimum cost factor 10. Plaintext passwords must never be logged or stored.
NFR-SEC-003	Security	Input Validation	All user inputs must be validated server-side using Spring Validation (@Valid, @NotNull). SQL injection and XSS prevented via Hibernate ORM parameterization.
NFR-SEC-004	Security	Data Encryption at Rest	Sensitive user credentials and database records stored in Oracle Database XE and MongoDB must use AES-256 encrypted tablespaces.
NFR-SCAL-001	Scalability	Concurrent Users	The system must support a minimum of 100 concurrent emergency officers without performance degradation, scalable via Docker container replicas.
NFR-REL-001	Reliability	Uptime & Availability	The system must achieve 99.5% uptime during evaluation. Eureka server handles automatic failover and client load balancing.
NFR-REL-002	Reliability	Structured Errors	All API endpoints must return structured JSON error responses (HTTP status + error message). No raw stack traces exposed to client.
NFR-USE-001	Usability	Mobile Responsiveness	All React UI screens must be fully functional on mobile devices with screen width ≥ 375px (iPhone SE size) for field officers.
NFR-USE-002	Usability	Accessibility (WCAG)	All UI components must meet WCAG 2.1 Level AA — color contrast ratio ≥ 4.5:1, semantic HTML5, and bilingual English/Tamil text.
NFR-MAINT-001	Maintainability	Layered Architecture	All microservices must follow standard Spring Boot layered architecture: Controller → Service → Repository with zero business logic in Controllers.
NFR-MAINT-002	Maintainability	API Documentation	All REST endpoints must be documented via Postman collections and OpenAPI / Swagger definitions.
NFR-COMP-001	Compliance	Data Privacy	Personal data collection must comply with the Digital Personal Data Protection (DPDP) Act 2023.

8. DATABASE DESIGN

8.1 Entity-Relationship Overview

ALERT 4.0 GOVERNMENT implements a hybrid dual-database architecture: **Oracle Database XE (11g/21c)** manages structured transactional entities (users, incident reports, alert broadcasts, approval workflows, and analytics snapshots). **MongoDB 6.0+** manages high-capacity binary photographic media evidence.

Key Relationships:

- DISTRICTS has a ONE-TO-MANY relationship with AUTH_USERS — one district employs many officers across taluks and villages.
- AUTH_USERS has a ONE-TO-MANY relationship with INCIDENT_REPORTS (via REPORTED_BY) — one VEO officer creates many incident reports.
- INCIDENT_REPORTS (Oracle XE) has a ONE-TO-ONE relationship with INCIDENT_MEDIA (MongoDB) via incidentId for photographic evidence.
- INCIDENT_REPORTS has a ONE-TO-MANY relationship with APPROVAL_WORKFLOWS — tracks multi-tier approval decisions (Taluk, District, Collector).
- ALERTS has a ONE-TO-MANY relationship with NOTIFICATIONS — one broadcast alert generates notification dispatch logs.
- DISTRICTS has a ONE-TO-MANY relationship with ANALYTICS_SNAPSHOTS — periodic analytics metrics stored per district.

8.2 Entity Descriptions (ENTITY 1: AUTH_USERS)

ENTITY: AUTH_USERS Service: auth-service Database: Oracle Database XE				
Column Name	Data Type	Constraints	Key	Description
ID	NUMBER(19) / BIGINT	PRIMARY KEY, AUTO_INC	PK	Unique user account identifier
USERNAME	VARCHAR2(100)	NOT NULL, UNIQUE	UK	Unique login username (e.g. VEO-COIMBATORESOUTH-1)
PASSWORD	VARCHAR2(255)	NOT NULL	—	BCrypt-hashed password string
ROLE	VARCHAR2(50)	NOT NULL	—	Role: village, taluk, district, collector, state
DISTRICT	VARCHAR2(100)	NULLABLE	—	Assigned administrative district name
TALUK	VARCHAR2(100)	NULLABLE	—	Assigned administrative taluk name
VILLAGE	VARCHAR2(100)	NULLABLE	—	Assigned village post name
NAME	VARCHAR2(150)	NULLABLE	—	Full display name of officer

8.2 Entity Descriptions (ENTITY 2: DISTRICTS & ENTITY 3: EMPLOYEES)

ENTITY: DISTRICTS Service: user-service Database: Oracle Database XE				
Column Name	Data Type	Constraints	Key	Description
ID	NUMBER(19) / BIGINT	PRIMARY KEY, AUTO_INC	PK	Unique district identifier
NAME	VARCHAR2(100)	NOT NULL, UNIQUE	UK	District name (e.g. Coimbatore, Chennai, Madurai)
TALUKS_JSON	CLOB / VARCHAR2(4000)	NOT NULL	—	JSON array of taluk names belonging to district
LATITUDE	NUMBER(10, 6)	NOT NULL	—	District headquarters latitude coordinate
LONGITUDE	NUMBER(10, 6)	NOT NULL	—	District headquarters longitude coordinate
TOTAL_TALUKS	NUMBER(5)	NOT NULL	—	Total count of taluks in district

ENTITY: EMPLOYEES Service: user-service Database: Oracle Database XE				
Column Name	Data Type	Constraints	Key	Description
ID	NUMBER(19) / BIGINT	PRIMARY KEY, AUTO_INC	PK	Unique employee record ID
USERNAME	VARCHAR2(100)	NOT NULL, UNIQUE	UK	References AUTH_USERS.USERNAME
EMAIL	VARCHAR2(150)	NOT NULL	—	Official government email address
PHONE	VARCHAR2(20)	NOT NULL	—	Emergency contact mobile number
DESIGNATION	VARCHAR2(100)	NOT NULL	—	Officer designation (e.g. Tahsildar, VEO)
STATUS	VARCHAR2(20)	NOT NULL	—	ACTIVE, ON_LEAVE, SUSPENDED

8.2 Entity Descriptions (ENTITY 4: ALERTS, ENTITY 5: TRANSLATIONS, ENTITY 6: INCIDENT_REPORTS)

ENTITY: ALERTS Service: alert-service Database: Oracle Database XE				
Column Name	Data Type	Constraints	Key	Description
ID	NUMBER(19) / BIGINT	PRIMARY KEY, AUTO_INC	PK	Unique alert broadcast identifier
TITLE	VARCHAR2(255)	NOT NULL	—	Emergency broadcast headline
MESSAGE	CLOB / VARCHAR2(4000)	NOT NULL	—	Detailed emergency instruction message
SEVERITY	VARCHAR2(50)	NOT NULL	—	CRITICAL, HIGH, MEDIUM, LOW
DISTRICT	VARCHAR2(100)	NOT NULL	—	Targeted district
TALUK	VARCHAR2(100)	NULLABLE	—	Targeted taluk (or Statewide)
STATUS	VARCHAR2(50)	NOT NULL	—	DRAFT, PENDING_APPROVAL, APPROVED, DISPATCHED
CREATED_AT	TIMESTAMP	NOT NULL	—	Alert creation timestamp

ENTITY: INCIDENT_REPORTS Service: incident-service Database: Oracle Database XE				
Column Name	Data Type	Constraints	Key	Description
ID	VARCHAR2(100)	PRIMARY KEY	PK	UUID string identifier for the incident
TITLE	VARCHAR2(255)	NOT NULL	—	Short summary title of the emergency
CATEGORY	VARCHAR2(50)	NOT NULL	—	Flood, Fire, Medical, Building Collapse, etc.
SEVERITY	VARCHAR2(50)	NOT NULL	—	Low, Medium, High, Severe, Extremely Severe
STATUS	VARCHAR2(50)	NOT NULL	—	Waiting for Taluk, Taluk Verified, Waiting for Collector, Resolved
DISTRICT	VARCHAR2(100)	NOT NULL	—	District location
TALUK	VARCHAR2(100)	NOT NULL	—	Taluk jurisdiction
REPORTED_BY	VARCHAR2(100)	NOT NULL	—	Username of reporting VEO officer

8.2 Entity Descriptions (ENTITY 7: INCIDENT_MEDIA & ENTITY 8: APPROVAL_WORKFLOWS)

ENTITY: INCIDENT_MEDIA Service: incident-service Database: MongoDB				
Field Name	BSON Type	Constraints	Key	Description
_id	ObjectId	PRIMARY KEY	PK	Unique MongoDB document identifier
incidentId	String	NOT NULL, INDEX	FK	References Oracle INCIDENT_REPORTS.ID
photoBase64	String / Binary	NOT NULL	—	Full photographic evidence in Base64 string format
uploadedAt	Date	NOT NULL	—	Timestamp of upload
contentType	String	NULLABLE	—	image/jpeg, image/png

ENTITY: APPROVAL_WORKFLOWS Service: approval-service Database: Oracle Database XE				
Column Name	Data Type	Constraints	Key	Description
ID	NUMBER(19) / BIGINT	PRIMARY KEY, AUTO_INC	PK	Unique workflow ID
INCIDENT_ID	VARCHAR2(100)	NOT NULL	—	Incident associated with approval request
APPROVAL_LEVEL	VARCHAR2(50)	NOT NULL	—	TALUK, DISTRICT, COLLECTOR
STATUS	VARCHAR2(50)	NOT NULL	—	PENDING, APPROVED, REJECTED, ESCALATED
APPROVER_NAME	VARCHAR2(100)	NULLABLE	—	Name of reviewing authority
DECISION_TIME	TIMESTAMP	NULLABLE	—	Timestamp of action
COMMENTS	VARCHAR2(500)	NULLABLE	—	Reviewer comments or rejection reason

8.2 Entity Descriptions (ENTITY 9: NOTIFICATIONS, ENTITY 10: ANALYTICS, ENTITY 11: AI_PREDICTIONS)

ENTITY: NOTIFICATIONS Service: notification-service Database: Oracle Database XE				
Column Name	Data Type	Constraints	Key	Description
ID	NUMBER(19) / BIGINT	PRIMARY KEY, AUTO_INC	PK	Unique notification identifier
USER_ID	VARCHAR2(100)	NOT NULL	—	Recipient user ID or role identifier
TITLE	VARCHAR2(255)	NOT NULL	—	Notification headline
MESSAGE	VARCHAR2(1000)	NOT NULL	—	Notification content
IS_READ	NUMBER(1) / BOOLEAN	DEFAULT 0	—	0 = Unread, 1 = Read
CREATED_AT	TIMESTAMP	NOT NULL	—	Timestamp of dispatch

ENTITY: ANALYTICS_SNAPSHOTS Service: analytics-service Database: Oracle Database XE				
Column Name	Data Type	Constraints	Key	Description
ID	NUMBER(19) / BIGINT	PRIMARY KEY, AUTO_INC	PK	Unique snapshot identifier
SNAPSHOT_DATE	DATE	NOT NULL	—	Date of analytics recording
TOTAL_INCIDENTS	NUMBER(10)	NOT NULL	—	Total incident count
ACTIVE_INCIDENTS	NUMBER(10)	NOT NULL	—	Active unverified/ongoing incidents
CRITICAL_ALERTS	NUMBER(10)	NOT NULL	—	Count of High / Severe / Critical incidents
AVG_RESPONSE_TIME	NUMBER(6, 2)	NOT NULL	—	Average verification time in minutes

ENTITY: AI_PREDICTIONS Service: ai-service Database: Oracle Database XE				
Column Name	Data Type	Constraints	Key	Description
ID	NUMBER(19) / BIGINT	PRIMARY KEY, AUTO_INC	PK	Unique AI inference record identifier
INCIDENT_ID	VARCHAR2(100)	NOT NULL, UNIQUE	UK	References INCIDENT_REPORTS.ID
PREDICTED_SEVERITY	VARCHAR2(50)	NOT NULL	—	Low, Medium, High, Severe, Extremely Severe
RECOMMENDATIONS	CLOB / VARCHAR2(4000)	NOT NULL	—	Ollama Llama 3 synthesized response advice
SPREAD_RADIUS_KM	VARCHAR2(50)	NULLABLE	—	Estimated hazard spread radius
CONFIDENCE_SCORE	NUMBER(5, 2)	NULLABLE	—	Model confidence percentage
PREDICTED_AT	TIMESTAMP	NOT NULL	—	Inference timestamp

9. API DESIGN OVERVIEW

9.1 REST API Endpoints Catalog

Auth Service (Port: 8082) & User Service (Port: 8083)			
Method	Endpoint	Auth Required	Description
POST	/api/v1/auth/login	No	Authenticates credentials; returns signed JWT and user profile.
POST	/api/v1/auth/register	No / Admin	Registers new officer; returns user record.
GET	/api/v1/users/me	Yes	Retrieves logged-in user profile details.
GET	/api/v1/reference/districts	Yes	Fetches 38 Tamil Nadu districts and taluks JSON.

Incident Service (Port: 8084) & Alert Service (Port: 8085)			
Method	Endpoint	Auth Required	Description
GET	/api/v1/incidents	Yes	Retrieves all active disaster incident reports.
POST	/api/v1/incidents	Yes	Creates new incident with metadata and Base64 photo.
PUT	/api/v1/incidents/{id}/status	Yes	Updates incident status (e.g. Taluk Verified, Resolved).
GET	/api/v1/incidents/{id}/image	Yes	Retrieves Base64 photographic evidence from MongoDB.
GET	/api/v1/alerts	Yes	Fetches active emergency broadcast alerts.
POST	/api/v1/alerts	Yes	Creates targeted public alert broadcast.

AI & Inference Service (FastAPI Port: 8000 & Spring Boot Port: 8089)			
Method	Endpoint	Auth Required	Description
POST	/ai/predict-risk	Internal	Ollama Llama 3 NLP risk scoring & bilingual recommendation.
GET	/api/v1/ai/weather-advisory	Yes	Open-Meteo meteorological threat advisory via Resilience4j.
GET	/health	No	FastAPI health check endpoint returning {"status":"ok"}.

10. DEVOPS & DEPLOYMENT PLAN

10.1 CI/CD Pipeline — Jenkins

Stage	Stage Name	Actions
1	Source Checkout	Jenkins pulls latest code from GitHub main branch via webhook on push.
2	Build	mvn clean package -DskipTests for all Spring Boot services; npm run build for React.
3	Unit Test	mvn test — executes JUnit 5 unit tests; fails pipeline if any test breaks.
4	Code Quality	SonarQube static analysis scanning for bugs, vulnerabilities, and code smells.
5	Docker Build	docker build -t [service-name]:latest for all 12 container images.
6	Deploy (Docker Compose)	docker-compose down && docker-compose up -d on target host.
7	Health Check	curl /actuator/health on each service — pipeline SUCCESS if all UP.

10.2 Docker Container Configuration

Container	Image / Context	Port Mapping	Depends On	Environment Variables
oracle-db	gvenzl/oracle-xe:slim-faststart	1522:1521	—	ORACLE_PASSWORD=admin
local-mongo	mongo:latest	27017:27017	—	volumes: mongo_data:/data/db
discovery-server	./discovery-server	8761:8761	—	EUREKA_CLIENT_REGISTERWITHEUREKA=false
api-gateway	./api-gateway	8081:8081	discovery-server	EUREKA_CLIENT_SERVICEURL_DEFAULTZONE
auth-service	./auth-service	8082:8082	discovery-server, oracle-db	SPRING_DATASOURCE_URL=jdbc:oracle:...
incident-service	./incident-service	8084:8084	discovery-server, oracle-db	SPRING_DATA_MONGODB_URI
alertgov-ai	./alertgov-ai	8000:8000	ollama	OLLAMA_URL=http://ollama:11434/api/generate
frontend	./alertgov-frontend	5173:80	api-gateway	VITE_API_URL=http://localhost:8081

10.3 Deployment Environments

Environment	Infrastructure	Purpose & Access
Development	localhost (Windows / Linux)	Individual workstations; hot-reload via Vite HMR and Spring DevTools.
Testing / QA	Shared Server / VM	Integration testing and mentor review; automated deployment via Jenkins.
Demo / Evaluation	Local Cluster / Docker Host	SIH live demonstration; stable, high-performance local evaluation.

11. PROJECT TIMELINE & MILESTONES

Week	Phase	Deliverables & Key Accomplishments
1	Foundation	✔ SRS document finalized ✔ Git repo with branching strategy ✔ Oracle XE & MongoDB schemas configured ✔ Eureka discovery server + API Gateway boilerplate operational.
2	Core Backend	✔ auth-service (JWT, BCrypt) complete ✔ user-service complete ✔ incident-service CRUD developed ✔ Spring Security filter chain verified end-to-end.
3	Full Backend	✔ alert-service & approval-service complete ✔ notification-service complete ✔ Postman collection created ✔ Inter-service Eureka load-balanced routing verified.
4	AI & Weather	✔ Python FastAPI service deployed on port 8000 ✔ Ollama Llama 3 bilingual inference connected ✔ Open-Meteo weather API with Resilience4j circuit breakers built in ai-service.
5	Frontend Complete	✔ React 19 + Vite bilingual portal built ✔ Leaflet.js interactive GIS mapping integrated ✔ Recharts analytics dashboard developed ✔ Multi-tier role portals connected.
6	Integration & Testing	✔ Full end-to-end verification pipeline (Village → Taluk → District → Collector → State) passing ✔ Docker Compose multi-container cluster validated ✔ Tested under 50+ concurrent queries.
7	Polish & Documentation	✔ Live dashboard sync and analytics chart rendering fixed ✔ Official PDF reporting integrated ✔ API Documentation & architecture docs created ✔ Internal Review demo successfully presented.
8	Final Submission	✔ Final SRS v2.0 published ✔ Git repo tagged v2.0 release ✔ Live demonstration & presentation slide deck finalized for SIH submission.

12. TEAM COMPOSITION & ROLE ASSIGNMENT

Member Name	Branch	Year	Key Responsibilities
Farah Hamna M	IT	III	Frontend Architecture (React 19, Leaflet GIS, Recharts, Bilingual UI), UI/UX Design, and Component Integration.
Rishikesh V	IT	III	Backend Lead (Spring Boot 3.2.4, Gateway, Eureka, JWT Security, Oracle XE & MongoDB persistence), DevOps & Docker Compose.
Shruthin K	IT	III	AI/ML Integration (Python FastAPI, Ollama Llama 3 bilingual reasoning, Resilience4j weather circuit breakers), Testing & API Docs.

Ownership Matrix

Component	Primary Owner	Secondary Owner	Support
Eureka & API Gateway	Rishikesh V	Farah Hamna M	Shruthin K
Core Microservices (Auth, Incident, Alert)	Rishikesh V	Shruthin K	Farah Hamna M
Approval & Notification Services	Rishikesh V	Farah Hamna M	Shruthin K
AI Microservice & Weather Service	Shruthin K	Rishikesh V	Farah Hamna M
React 19 Frontend & GIS Maps	Farah Hamna M	Rishikesh V	Shruthin K
Database Schemas (Oracle XE + Mongo)	Rishikesh V	Farah Hamna M	Shruthin K
DevOps & Docker Deployment	Rishikesh V	Shruthin K	Farah Hamna M
SRS Documentation & Evaluation	All Members	All Members	All Members

Interdisciplinary Contribution Map

Stream	Expected Contribution Area	Specific Contribution in THIS Project
Information Technology (IT)	Full Stack Java (Spring Boot), Microservices, AI/ML Integration, React Frontend, DevOps	Engineered all 11 Spring Boot microservices, React 19 GIS portal, Ollama AI microservice, and Oracle XE/MongoDB dual persistence.
Domain & Governance	Disaster workflow design, SOP compliance, TNDMA policy alignment	Structured the 5-tier disaster verification hierarchy according to official Tamil Nadu Emergency Operations Center guidelines.

13. RISK REGISTER

ID	Risk Description	Prob.	Impact	Mitigation Strategy
R01	Ollama local LLM inference latency exceeds 5 seconds during high-load disasters	M	H	Implemented rule-based fallback heuristic analysis in FastAPI that immediately supplies valid severity ratings and advice if LLM timeout occurs.
R02	Open-Meteo external weather API outage or network disruption	M	H	Wrapped endpoint in Resilience4j CircuitBreaker with automated fallback to default Tamil Nadu seasonal advisory data.
R03	Oracle Database XE connection pool exhaustion under multi-service startup	M	H	Tuned HikariCP connection pool limits (maximum-pool-size=3) across microservices to maintain lightweight concurrent connection footprint.
R04	Large photographic evidence uploads overloading database performance	M	M	Decoupled persistence: metadata stored in Oracle XE, while binary Base64 images are stored in dedicated MongoDB document collections.
R05	Unauthorized alert dissemination causing public panic	L	H	Enforced strict multi-level approval workflow; broadcast alerts cannot be published without District Collector authorization.
R06	Loss of internet connectivity in remote disaster zones	M	M	Client-side local storage caching in React frontend allows offline draft capture with automatic sync upon reconnection.
R07	Accidental data exposure or credential leaks	L	H	Stateless JWT token verification on all protected endpoints; BCrypt password hashing; zero secrets committed to Git repositories.
R08	Inaccurate or unverified disaster reports submitted from field	M	H	Mandatory Tahsildar verification step in Taluk Command Center before incidents can escalate to District EOC.

14. APPENDIX

14.1 Glossary of Domain-Specific Terms

Term	Definition
TNDMA	Tamil Nadu Disaster Management Authority — State nodal agency responsible for disaster prevention, mitigation, and emergency response.
VEO	Village Emergency Officer — Ground-level reporting authority stationed at village panchayat emergency posts.
DEOC	District Emergency Operations Centre — 24/7 command center operating at the District Collectorate.
Tahsildar	Administrative revenue and executive officer heading a Taluk, responsible for incident verification.

14.2 AI Model Details — Training & Runtime Logs

- **Runtime Model:** Meta Llama 3 8B Instruct (4-bit quantized GGUF) hosted on Ollama local server.
- **Execution Environment:** Python 3.13, FastAPI 0.141.1, Uvicorn 0.52.1 on port 8000.
- **Performance:** Average inference latency: 2.1 seconds; Heuristic fallback execution time: 18ms.
- **Target Languages:** English & Tamil (தமிழ் script output verified).

14.3 API Request-Response Samples

```
POST /api/v1/incidents
Request: {"title": "Flash Flooding at Peelamedu", "category": "Flood", "severity": "High", "district": "Coimbatore", "taluk": "Coimbatore South"}
Response: {"id": "INC-7a89f214", "status": "Waiting for Taluk", "reportedAt": "2026-09-20T19:30:00.000Z"}
```

14.4 References & Citations

- Open-Meteo Weather API: <https://open-meteo.com/>
- TNDMA Standard Operating Procedures (SOP 2024–2026), Government of Tamil Nadu.
- Meta Llama 3 Model Card & Technical Specifications, Meta AI (2024).